

# Complete Instructor Guide

This guide explains how to understand, prepare, teach, and assess the ANL559 Studio Refresh. It is written for an instructor who is new to the redesigned course and needs a practical manual, not only a slide deck.

## 1. Course Design In One Page

ANL559 is taught as a six-week analytics automation studio. Students do not mainly learn AI agents by listening to theory. They learn by supervising an AI agent through realistic analytics work.

The course rhythm is:

1. **Build:** use an AI agent to produce a useful first version.
2. **Break:** deliberately expose defects, weak assumptions, missing evidence, or unsafe claims.
3. **Defend:** explain what can be trusted, what was rejected, and what still should not be shipped.

This rhythm repeats every week across different analytics automation tasks:

| Week | Mission                   | Professional capability                                   |
|------|---------------------------|-----------------------------------------------------------|
| 1    | Reporting fire drill      | Turn recurring reporting into a checked workflow.         |
| 2    | Board number disagreement | Reconcile data and build quality gates.                   |
| 3    | Limited retention budget  | Turn modelling into a cost-aware decision.                |
| 4    | Notebook to tool          | Operationalise analytics with inputs, tests, and runbook. |
| 5    | Executive question        | Build and evaluate a source-grounded analyst agent.       |
| 6    | Launch review             | Defend launch readiness under incident pressure.          |

The underlying message to students is simple:

The agent can accelerate production, but the human remains responsible for evidence, judgement, and operational limits.

## 2. What Makes This Design Different

The redesign is mission-first, not framework-first. Each seminar starts with a business problem, then introduces only the concepts needed to solve that problem.

This is deliberate:

- Students need motivation before terminology.
- AI-agent work is best learned through supervision habits.
- Real analytics automation fails at definitions, joins, leakage, validation, unsupported claims, and ownership.
- A course about automation should produce artefacts, not only reports about automation.

The course still teaches governance, evaluation, risk, and quality, but it teaches them through practical artefacts:

- Metric definitions.
- Row-count bridges.
- Leakage audits.
- Input contracts.
- Evaluation cases.
- Runbooks.
- Launch checklists.

## 3. Package Map

Use the package as follows:

| Resource                     | Primary user | Purpose                                                     |
|------------------------------|--------------|-------------------------------------------------------------|
| README.md                    | Instructor   | Folder map and quick start.                                 |
| Studio_Course_Blueprint.md   | Instructor   | Six-week course arc and seminar structure.                  |
| Complete_Instructor_Guide.md | Instructor   | Full course design and delivery manual.                     |
| Instructor_Delivery_Notes.md | Instructor   | Week-by-week teaching notes.                                |
| Facilitation_Guide.md        | Instructor   | Teaching moves, roles, inclusion, and classroom operations. |

| Resource                                        | Primary user            | Purpose                                                          |
|-------------------------------------------------|-------------------------|------------------------------------------------------------------|
| <code>Student_Workbook.md</code>                | Students                | Weekly student instructions and evidence expectations.           |
| <code>Mission_Cards.md</code>                   | Students                | Compact weekly mission briefs.                                   |
| <code>Exercises/</code>                         | Instructor and students | Detailed weekly exercises, timings, prompts, and solution notes. |
| <code>Worked_Examples.md</code>                 | Instructor and students | Models of strong evidence.                                       |
| <code>Prompt_Guidelines_and_Library.md</code>   | Instructor and students | Agent work-order patterns and repair prompts.                    |
| <code>Sample_Data_and_Scenarios.md</code>       | Instructor and students | Fictional company, data context, known checks, and defects.      |
| <code>Starter_Kit/</code>                       | Students                | Templates, sample data, and example artefacts.                   |
| <code>Slide_Decks/</code>                       | Instructor              | Six 77-slide seminar decks.                                      |
| <code>Portfolio_and_Assessment_Models.md</code> | Instructor and markers  | Rubric, assessment shape, and viva prompts.                      |

## 4. How To Prepare Before The Course Starts

### Instructor Preparation

Before Week 1:

1. Read `Studio_Course_Blueprint.md`.
2. Read this guide fully.
3. Skim all six decks to understand the repeated structure.
4. Open `Student_Workbook.md` and decide how students will receive it.
5. Confirm that `Starter_Kit/sample_data/` opens correctly on your machine.
6. Decide what AI-agent surface students will use.
7. Decide whether teams submit through a Git repo, LMS folder, shared drive, or zipped portfolio.
8. Prepare a class “defect wall” using a whiteboard, shared document, Padlet, Miro, Teams channel, or LMS discussion thread.

## Technical Preparation

Recommended:

- Python 3.10 or later.
- `pandas`.
- `pytest`.
- Jupyter, VS Code, or another familiar environment.
- Git or a simple shared-folder workflow.
- AI-agent access for each team.

If not all students can code:

- Use spreadsheets for Weeks 1 and 2.
- Let students audit agent transcripts.
- Let students write tests as plain-language acceptance checks.
- Assign non-coding roles: Reviewer and Reporter.

## Team Setup

Teams of two or three work best.

Use rotating roles:

- **Driver:** operates files, repo, spreadsheet, notebook, tool, and agent.
- **Reviewer:** checks assumptions, validates outputs, and challenges the agent.
- **Reporter:** maintains evidence, AI-use log, defence note, and demo narrative.

Role rotation matters. Without rotation, the fastest coder may learn the most while others become observers.

## 5. How To Use The 77-Slide Decks

Each weekly deck is intentionally long enough for a three-hour seminar. It is not meant to be delivered as 77 lecture slides.

Use the deck as a teaching spine:

- Some slides are shown for 20 seconds as transitions.
- Some slides stay on screen for 10-20 minutes during activities.
- Some slides are backup guidance for weaker classes.
- Some slides are skipped if the room is moving well.

The deck structure is:

| Slide block         | Teaching purpose                                |
|---------------------|-------------------------------------------------|
| Opening and mission | Create business urgency.                        |
| Start here          | Name files, assumptions, and starter materials. |
| Worked example      | Show the evidence standard.                     |
| Concept depth       | Teach only the concepts needed today.           |
| Prompt clinic       | Improve agent instructions before building.     |
| Guided practice     | Make students rehearse the thinking.            |
| Sprint A            | Build the first version.                        |
| Break moment        | Release failure cards and expose defects.       |
| Sprint B            | Repair, harden, and create evidence.            |
| Peer review         | Make evidence inspectable by others.            |
| Defence             | Convert activity into assessable judgement.     |
| Close               | Submit, reflect, and connect to next week.      |

### Slide Delivery Rule

Do not treat every slide as a mini-lecture. A good 3-hour seminar has:

- 25-35 minutes of instructor explanation.
- 30-45 minutes of guided discussion and practice.
- 70-90 minutes of student build, break, repair, and peer review.
- 15-25 minutes of evidence writing and closure.

If you are speaking for more than 15 minutes without students producing something, move to an activity slide.

## 6. Standard Three-Hour Delivery Plan

Use this pacing as the default:

|  | Time  | Activity                  | Instructor role                               |
|--|-------|---------------------------|-----------------------------------------------|
|  | 0-5   | Retrieval link            | Connect last week to this week.               |
|  | 5-15  | Cold open                 | Create urgency through the stakeholder scene. |
|  | 15-25 | Mission and starter files | Explain files, artefact, and assumptions.     |
|  | 25-40 | Worked example            | Show what strong evidence looks like.         |

|  | Time    | Activity                          | Instructor role                           |
|--|---------|-----------------------------------|-------------------------------------------|
|  | 40-55   | Concept burst                     | Teach the three necessary concepts.       |
|  | 55-75   | Prompt clinic and guided practice | Improve work orders before agent use.     |
|  | 75-105  | Sprint A                          | Students build first version.             |
|  | 105-125 | Break moment                      | Release failure cards and expose defects. |
|  | 125-155 | Sprint B                          | Students repair and collect evidence.     |
|  | 155-170 | Peer demo                         | Teams inspect each other's evidence.      |
|  | 170-178 | Defence note                      | Students write the claim they can defend. |
|  | 178-180 | Exit ticket                       | Collect habit and unresolved risk.        |

The exact timing can flex. The sequence should not.

## 7. Core Teaching Moves

### Cold Open

Start with the business situation, not definitions.

Example:

“The COO wants the KPI pack by 10:00. The analyst who built last week’s version is away. What would make the report safe to send?”

The purpose is to make students want the concept before you teach it.

### Worked Example

Show a short, concrete model of good work.

Ask:

- What claim is being made?
- Where is the evidence?
- What was rejected?
- What limitation is named?

Avoid using worked examples as answer keys. Use them as examples of evidence structure.

### **Prompt Clinic**

Students often assume prompting is about clever wording. In this course, prompting is about supervision.

A good work order includes:

- Goal.
- Business context.
- Inputs.
- Definitions.
- Acceptance criteria.
- Constraints.
- Output format.

### **Failure Cards**

Failure cards make risk visible and productive. Release them after the first build, not before.

This matters because students first need to experience how plausible the first draft can look.

### **Defect Wall**

The defect wall is a public list of failures found by teams.

Each defect should include:

- Team name.
- Failure type.
- Evidence that exposed it.
- Repair or next step.

The defect wall changes classroom culture. Students learn that finding failure is success.

## Defence Note

The defence note turns work into learning. It forces students to state judgement.

A good defence note says:

- I trust this because...
- I rejected this because...
- The remaining risk is...
- The next action is owned by...

## 8. How To Coach AI-Agent Use

Students should not be rewarded for accepting agent output quickly. They should be rewarded for supervising agent output well.

Use these coaching questions:

- What exactly did you ask the agent to do?
- Which definition did the agent assume?
- Which output did you reject?
- What evidence changed your mind?
- What test would catch this next time?
- What should the agent not be allowed to decide?

## Common Agent Failure Modes

| Failure mode           | Typical sign                                                 | Teaching response                 |
|------------------------|--------------------------------------------------------------|-----------------------------------|
| Invented definition    | Metric seems plausible but differs from approved definition. | Return to metric contract.        |
| Silent row loss        | Join output is clean but source counts shrink.               | Build row-count bridge.           |
| Leakage                | Model score is suspiciously strong.                          | Audit decision-time availability. |
| Happy-path tool        | Works only for valid input.                                  | Add bad-input tests.              |
| Unsupported claim      | Answer sounds confident but source does not support it.      | Check citation support.           |
| Overbroad launch claim | Demo works but ownership or limits are unclear.              | Use launch checklist.             |



## 9. How To Assess The Course

Assessment should reward evidence and judgement, not polish alone.

Use the same four criteria every week:

1. **Useful artefact:** Does it run, inspect, or solve the mission?
2. **Agent supervision:** Did students accept, reject, modify, and verify agent output?
3. **Break evidence:** Did students expose a meaningful failure?
4. **Defence:** Can students explain the claim, evidence, and limit?

Strong evidence can be simple:

- A table of metric definitions.
- A failing and passing test.
- A row-count bridge.
- A leakage audit.
- A rejected bad-input request.
- A citation-support table.
- A launch checklist.

Weak evidence is vague:

- “We used AI.”
- “The model is accurate.”
- “The answer has citations.”
- “The tool works.”
- “The system is ready.”

## 10. Weekly Delivery Guide

### Week 1 - Reporting Fire Drill

**Teaching goal:** Students learn that automation means reproducibility and checks, not a polished chart.

**Key concept:** Metric definition as contract.

**What to emphasise:**

- The agent must not define the metric.
- Smoke tests are part of the artefact.
- AI-use logs should record a specific accepted or rejected output.

**Best activity:** Prompt rewrite and smoke-test hunt.

**Common mistake:** Students report “active customers” without specifying whether it means paid active account or recent login.

**Instructor intervention:** Ask, “Would a cancelled customer who logged in last week count?”

**Minimum evidence:** Metric definition, one failing check, one corrected agent assumption.

## Week 2 - The Board Number Is Wrong

**Teaching goal:** Students learn that trusted numbers require reconciliation logic.

**Key concept:** Row-count bridge.

**What to emphasise:**

- Inner joins can hide problems.
- Quarantine is more honest than silent cleaning.
- Business definitions matter as much as technical joins.

**Best activity:** Join choice debate and quarantine table.

**Common mistake:** Students choose the cleanest table rather than the appropriate source of truth.

**Instructor intervention:** Ask, “Which records disappeared, and should they have disappeared?”

**Minimum evidence:** Source counts, join coverage, at least one quarantined record.

## Week 3 - Limited Retention Budget

**Teaching goal:** Students learn that model performance must be translated into decision policy.

**Key concept:** Leakage and threshold economics.

**What to emphasise:**

- A model score is not a decision.
- Baselines are required.
- Features must be available at decision time.
- Thresholds should reflect cost or capacity.

**Best activity:** Leakage hunt and threshold economics.

**Common mistake:** Students include `post_offer_response` because it improves model performance.

**Instructor intervention:** Ask, “Would this column exist before the offer decision?”

**Minimum evidence:** Leakage audit and threshold rationale.

#### **Week 4 - From Notebook to Tool**

**Teaching goal:** Students learn that operational analytics needs boundaries, tests, and ownership.

**Key concept:** Input contract.

**What to emphasise:**

- A notebook is not yet a tool.
- Bad inputs are part of product design.
- Runbooks are evidence of operational thinking.

**Best activity:** Bad-input lab and handoff test.

**Common mistake:** Students only test the happy path.

**Instructor intervention:** Ask, “What happens when `customer_id` is missing?”

**Minimum evidence:** One valid request and one rejected bad request.

#### **Week 5 - The Executive Question**

**Teaching goal:** Students learn that analyst agents need source grounding, uncertainty, and refusal.

**Key concept:** Evidence-supported answer.

**What to emphasise:**

- A citation must support the exact claim.
- Association is not causation.
- Refusal is a valid analyst behaviour.

**Best activity:** Citation support check and refusal rehearsal.

**Common mistake:** Students treat any source mention as support.

**Instructor intervention:** Ask, “Which sentence in the source supports that exact sentence?”

**Minimum evidence:** One unsupported claim caught by an evaluation case.

## **Week 6 - Launch Review**

**Teaching goal:** Students learn to make a launch recommendation under evidence and incident pressure.

**Key concept:** Scoped launch decision.

**What to emphasise:**

- Launch is not a feature tour.
- Limited pilot or no-ship can be a strong recommendation.
- Every claim needs evidence, owner, and limit.

**Best activity:** Incident drill and viva circle.

**Common mistake:** Students overclaim from a polished demo.

**Instructor intervention:** Ask, “What should not ship yet?”

**Minimum evidence:** Ship / no-ship decision, evidence map, incident response, owner.

## **11. Managing Different Student Skill Levels**

### **For Strong Coders**

Push them beyond implementation:

- Ask for clearer tests.
- Ask for better evidence.
- Ask for a stronger defence note.
- Ask them to review another team’s artefact.

### **For Weaker Coders**

Do not let coding become the whole course.

Give them valid roles:

- Write acceptance criteria.
- Check row counts.
- Review feature leakage.
- Evaluate citation support.
- Prepare defence notes.
- Run peer review.

## **For Teams With Agent Access Problems**

Use fallback tasks:

- Audit a provided agent transcript.
- Write a work order and acceptance tests.
- Review a worked example and identify missing evidence.
- Produce the defence note from a supplied artefact.

## **12. How To Keep The Seminar Engaging**

Use pace changes:

- Scene.
- Question.
- Worked example.
- Short concept.
- Timed task.
- Defect reveal.
- Repair sprint.
- Peer challenge.
- Defence.

Avoid long uninterrupted explanation. The strongest parts of the course happen when students are making decisions under constraints.

Use phrases that reinforce the culture:

- “Fast first draft. Slow acceptance.”
- “Show me the evidence.”
- “What did the agent get wrong?”
- “What should not be trusted yet?”
- “A limited pilot is a decision, not a failure.”

## **13. Troubleshooting**

### **Students Are Stuck At Setup**

Move them to a fallback:

- Spreadsheet mode.
- Transcript audit.
- Reviewer role.
- Plain-language test design.

Do not let setup failure consume the learning objective.

### **Students Trust The Agent Too Much**

Release a failure card and ask them to prove the output is safe.

### **Students Are Too Focused On Polished Slides**

Redirect to evidence:

- Where is the failing check?
- What changed after repair?
- What claim can you defend?

### **Teams Finish Too Early**

Use stretch options:

- Add monitoring.
- Add a second bad-input test.
- Add an adversarial evaluation case.
- Improve the runbook.
- Review another team's artefact.

### **Seminar Is Running Late**

Protect these blocks:

- Failure card.
- Repair.
- Defence note.

If needed, shorten concept explanation and peer demo. Do not skip the defence note.

## **14. After-Class Workflow**

After each seminar, ask students to submit:

- Artefact.
- Evidence item.
- AI-use-log entry.
- Defence note.

Instructor quick review:

1. Does the artefact exist?
2. Is there a named failure?
3. Is there evidence of repair or judgement?
4. Is the claim appropriately scoped?

Use the next week's retrieval link to discuss one strong example and one common issue.

## 15. Instructor Checklist

Before class:

- Read the weekly section in this guide.
- Open the weekly slide deck.
- Open the weekly exercise sheet.
- Confirm starter files are available.
- Choose one worked example to show.
- Prepare three failure cards.
- Decide how teams will submit evidence.

During class:

- Keep the mission visible.
- Keep concept teaching short.
- Start the timer for activities.
- Visit each team at least twice.
- Ask “what did you reject?” repeatedly.
- Put defects on the defect wall.
- End with a defence note.

After class:

- Check evidence quickly.
- Select one student artefact to discuss next week.
- Note the most common misconception.
- Adjust next week's retrieval link.

## 16. What Good Delivery Feels Like

A strong seminar feels active, practical, and slightly time-pressured. Students should not leave thinking that the course was about “using AI to do assignments faster.” They should leave thinking:

- I can scope an agent task.
- I can catch an agent failure.
- I can make analytics automation more reliable.
- I can defend what should and should not be trusted.

That is the course design.